Real-Time IoT Ingestion

```python
@app.route("/api/sensor-ingest", methods=["POST"])
def sensor_ingest():
    global latest_pi_data
    try:
        data = request.get_json()

        latest_pi_data['temperature'] = data.get('temperature', latest_pi_data['temperature'])
        latest_pi_data['humidity'] = data.get('humidity', latest_pi_data['humidity'])
        latest_pi_data['moisture'] = data.get('soil', data.get('moisture', latest_pi_data['moisture']))
        latest_pi_data['irradiance'] = data.get('solar_output', data.get('irradiance', latest_pi_data['irradiance']))
        latest_pi_data['light'] = data.get('light_raw', data.get('light', latest_pi_data['light']))

        print(f"🎉 [HTTP INGEST] {latest_pi_data}")

        # Store in history
        sensor_history.append({
            **latest_pi_data,
            "timestamp": datetime.now().isoformat()
        })
        if len(sensor_history) > 500:
            sensor_history.pop(0)

        # Emit to all connected dashboards
        socketio.emit('sensor-update', {
            "temperature": latest_pi_data['temperature'],
            "humidity": latest_pi_data['humidity'],
            "soil": latest_pi_data['moisture'],
            "solar_output": latest_pi_data['irradiance'],
            "light_raw": latest_pi_data['light'],
            "timestamp": datetime.now().isoformat()
        })

        return jsonify({"success": True}), 200

    except Exception as e:
        print(f"Sensor ingest error: {str(e)}")
        return jsonify({"error": str(e)}), 500
```

- **Function:** /api/sensor-ingest
- **What it does:** Acts as the high-speed bridge between the field sensors and the web dashboard.
- **Key Technical Points:**
    - **Data Reception:** Instantly receives JSON payloads (temperature, humidity, moisture, light) forwarded by the Raspberry Pi edge gateway.
    - **Memory Management:** Efficiently maintains a rolling buffer of the last 500 readings in the server's RAM for quick historical charting.
    - **Zero-Latency Updates:** Uses WebSockets (socketio.emit) to push new data directly to the React frontend in milliseconds, eliminating the need for the user to refresh the page.

AI Engine Initialization

```python
MODEL_PATH = 'model_leaf_disease.h5'
model_irr = joblib.load("models/model_irrigation.pkl")
model_solar = joblib.load("models/model_solar_output.pkl")
model_crop = joblib.load("models/model_crop_health.pkl")
encoder_crop = joblib.load("models/encoder_crop_health.pkl")

model = None
try:
    if os.path.exists(MODEL_PATH):
        model = tf.keras.models.load_model(MODEL_PATH)
        print("✅ Model loaded successfully!")
        print(f"📐 Model INPUT shape: {model.input_shape}")
        print(f"📊 Model OUTPUT shape: {model.output_shape}")

        assert model.output_shape[-1] == len(CLASS_NAMES), \
            f"Model outputs {model.output_shape[-1]} classes, but labels = {len(CLASS_NAMES)}"

    else:
        print(f"❌ CRITICAL ERROR: Model file not found at '{os.path.abspath(MODEL_PATH)}'")
except Exception as e:
    print(f"❌ CRITICAL ERROR: Failed to load Keras model. Error: {str(e)}")
```

- **Function:** Model Loading Pipeline
- **What it does:** Boots up the multi-model Artificial Intelligence engine when the server starts.
- **Key Technical Points:**
    - **Multi-Framework Integration:** Seamlessly loads traditional Scikit-Learn machine learning models (via joblib) alongside a complex Deep Learning CNN (via TensorFlow/Keras).
    - **Predictive Analytics:** Initializes the brains behind the Irrigation Prediction, Solar Output Estimation, and Crop Health Scoring features.
    - **Safety Validations:** Includes built-in assert checks to guarantee the neural network's output layer perfectly matches the 50 hardcoded plant disease classes before allowing the server to run.

Computer Vision Preprocessing

```python
# --- Image Preprocessing ---
def preprocess_leaf_image(image_bytes):
    MODEL_INPUT_SIZE = (128, 128)

    img = Image.open(io.BytesIO(image_bytes))
    if img.mode != "RGB":
        img = img.convert("RGB")

    img = img.resize(MODEL_INPUT_SIZE)
    img_array = np.array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)
    return img_array
```

- **Function:** preprocess_leaf_image(image_bytes)
- **What it does:** Translates raw user-uploaded photos into mathematical data that the neural network can "see."
- **Key Technical Points:**
    - **Standardization:** Catches raw image bytes, enforces a standard RGB color space, and resizes every image to exactly 128x128 pixels.
    - **Array Transformation:** Converts the image into a NumPy array, which is the required mathematical structure for deep learning operations.
    - **Normalization:** Scales all pixel values down from a 0-255 range to a 0.0-1.0 range, drastically improving the neural network's prediction accuracy and speed.